JAVA FUNDAMENTALS

Complete Interview Preparation Guide

(Excluding OOP Topics)

Topics Covered

- Java Basics & JVM Architecture
- Data Types, Variables & Type Casting
- Operators & Control Flow
- Arrays & Strings
- Exception Handling
- Collections Framework
- Generics
- Multithreading & Concurrency
- Java 8+ Features (Lambdas, Streams)
- File I/O & Serialization
- Memory Management & Garbage Collection
- Frequently Asked Cheat Sheet

Prepared for Technical Interview Success

TABLE OF CONTENTS

1. Java Basics & JVM Architecture....3
1. Java Basics & JVM Architecture....3
1.1 Core Java Concepts....3
1.1 Core Java Concepts....3
2. Data Types, Variables & Type Casting....5
2. Data Types, Variables & Type Casting....5
2.1 Primitive Data Types....5
2.1 Primitive Data Types....5
3. Operators & Control Flow....7
3. Operators & Control Flow....7
3.1 Operators....7
3.1 Operators....7
3.2 Control Flow....7
3.2 Control Flow....7
4. Arrays & Strings....9
4. Arrays & Strings....9
4.1 Arrays....9
4.1 Arrays....9
4.2 Strings....9
4.2 Strings....9
5. Exception Handling....11
5. Exception Handling....11
5.1 Exception Hierarchy & Types....11
5.1 Exception Hierarchy & Types....11
6. Collections Framework....13
6. Collections Framework....13
6.1 Collections Overview....13
6.1 Collections Overview....13
7. Generics....15
7. Generics....15
8. Multithreading & Concurrency....16
8. Multithreading & Concurrency....16
8.1 Thread Basics....16
8.1 Thread Basics....16
9. Java 8+ Features....18
9. Java 8+ Features....18
9.1 Lambda Expressions & Functional Interfaces....18
9.1 Lambda Expressions & Functional Interfaces....18
10. File I/O & Serialization....20
10. File I/O & Serialization....20
11. Memory Management & Garbage Collection....21
11. Memory Management & Garbage Collection....21
12. Frequently Asked Interview Cheat Sheet ★....22
12. Frequently Asked Interview Cheat Sheet ★....22

1. Java Basics & JVM Architecture

1.1 Core Java Concepts

Q: What is Java and what are its key features?

Answer:

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now Oracle) in 1995. Key features include:

- **Platform Independence (WORA):** Write Once, Run Anywhere via bytecode and JVM
- **Strongly Typed:** Every variable must have a declared type
- **Automatic Memory Management:** Garbage collector handles memory deallocation
- **Multithreading:** Built-in support for concurrent programming
- **Security:** Bytecode verification, security manager, no explicit pointers
- **Rich Standard Library:** Comprehensive Java API (java.util, java.io, java.net, etc.)
- **Robust:** Strong exception handling and type checking at compile and runtime

Q: Explain the difference between JDK, JRE, and JVM.

Answer:

Component	Full Form	Purpose
JVM	Java Virtual Machine	Executes bytecode; platform-specific runtime engine
JRE	Java Runtime Environment	JVM + standard libraries; needed to run Java apps
JDK	Java Development Kit	JRE + compiler (javac) + tools; needed to develop Java apps

> **Note:** JDK contains JRE which contains JVM. You only need JRE to run; you need JDK to compile and develop.

Q: How does Java achieve platform independence?

Answer:

Java achieves platform independence through a two-step process:

- **Compilation:** javac compiles .java source files into platform-neutral bytecode (.class files)
- **Execution:** The JVM on each platform interprets or JIT-compiles the bytecode into native machine code

The bytecode is identical regardless of OS, but each OS has its own JVM implementation that translates it to native instructions.

```
Source (.java) → javac → Bytecode (.class) → JVM → Native Machine Code
```

Q: What is the JVM architecture? Explain its components.

Answer:

The JVM consists of the following major components:

- **Class Loader Subsystem:** Loads, links, and initializes class files
  - Bootstrap ClassLoader → Extension ClassLoader → Application ClassLoader

- **Runtime Data Areas:**
  - Method Area: Stores class metadata, static variables, constant pool
  - Heap: Object instances and arrays; managed by Garbage Collector
  - Java Stack: Per-thread; stores stack frames (local variables, operand stack)
  - PC Register: Tracks current instruction per thread
  - Native Method Stack: Supports native (C/C++) method calls
- **Execution Engine:**
  - Interpreter: Executes bytecode line by line (slow)
  - JIT Compiler: Compiles frequently used bytecode to native code (fast)
  - Garbage Collector: Reclaims unused heap memory
- **Native Method Interface (JNI):** Bridge between Java and native libraries

Q: What is the difference between heap and stack memory in Java?

Answer:

Aspect	Stack	Heap
Stores	Primitive values, references, method frames	All objects and arrays
Scope	Thread-local (each thread has own stack)	Shared across all threads
Size	Smaller, fixed size (configurable)	Larger, grows dynamically
Management	Auto (LIFO — method call/return)	GC managed
Exception	StackOverflowError	OutOfMemoryError
Access Speed	Faster	Slower

Q: What is Just-In-Time (JIT) compilation?

Answer:

JIT compilation is an optimization technique where the JVM's execution engine compiles frequently executed bytecode segments (hot spots) into native machine code at runtime. This eliminates repeated interpretation overhead.

- **HotSpot JVM:** Oracle's JVM uses HotSpot JIT which identifies hot methods/loops
- **C1 Compiler:** Client compiler — fast compilation with basic optimizations
- **C2 Compiler:** Server compiler — slower compilation but aggressive optimizations
- **Tiered Compilation:** Java 8+ default; starts with C1, promotes hot code to C2

> **Note:** JIT is why Java performance can match or exceed C++ for long-running applications.

2. Data Types, Variables & Type Casting

2.1 Primitive Data Types

Q: What are the 8 primitive data types in Java?

Answer:

Type	Size	Range/Default	Example
byte	8 bits	-128 to 127 / 0	`byte b = 100;`
short	16 bits	-32,768 to 32,767 / 0	`short s = 1000;`
int	32 bits	-2^31 to 2^31-1 / 0	`int i = 42;`
long	64 bits	-2^63 to 2^63-1 / 0L	`long l = 123L;`
float	32 bits	~7 decimal digits / 0.0f	`float f = 3.14f;`
double	64 bits	~15 decimal digits / 0.0	`double d = 3.14;`
char	16 bits	0 to 65,535 (Unicode) / \u0000	`char c = 'A';`
boolean	1 bit	true or false / false	`boolean b = true;`

Q: What is the difference between primitive types and wrapper classes?

Answer:

- **Primitive types:** int, double, boolean — stored on stack, no methods, cannot be null
- **Wrapper classes:** Integer, Double, Boolean — objects on heap, have utility methods, can be null
- **Autoboxing:** Automatic conversion of primitive to wrapper (int → Integer)
- **Unboxing:** Automatic conversion of wrapper to primitive (Integer → int)

```
Integer x = 5;         // autoboxing
int y = x;             // unboxing
Integer a = 127; Integer b = 127; // a == b → true (cache)
Integer c = 128; Integer d = 128; // c == d → false (no cache!)
```

 Note: Integer cache: JVM caches Integer objects from -128 to 127. Comparing with == outside this range gives false. Always use .equals() for wrapper comparisons.

Q: Explain type casting in Java — implicit vs explicit.

Answer:

- **Widening (Implicit):** Smaller → larger type; no data loss; done automatically

```
int i = 10; long l = i;  // widening — implicit
// byte → short → int → long → float → double
```

- **Narrowing (Explicit):** Larger → smaller type; possible data loss; requires cast operator

```
double d = 9.99; int i = (int) d;  // i = 9 (truncated, not rounded)
```

- **String Conversions:** Use Integer.parseInt(), String.valueOf(), etc.

```
int x = Integer.parseInt("42");
String s = String.valueOf(42);  // or Integer.toString(42)
```

Q: What is the difference between == and .equals() in Java?

Answer:

- **== operator:** Compares references (memory addresses) for objects; compares values for

primitives

- **.equals():** Compares logical/semantic content of objects (can be overridden)

```
String s1 = new String("hello");
String s2 = new String("hello");
s1 == s2       // false (different objects)
s1.equals(s2)  // true (same content)
String s3 = "hello"; String s4 = "hello";
s3 == s4       // true (String pool — same literal reference)
```

> **Note:** Always override both hashCode() and equals() together. If two objects are equal, they must have the same hashCode.

3. Operators & Control Flow

3.1 Operators

Q: What are the types of operators in Java?

Answer:

- **Arithmetic:** + - * / % (modulo) ++ -- (increment/decrement)
- **Relational:** == != > < >= <=
- **Logical:** && (AND) || (OR) ! (NOT) — short-circuit evaluation
- **Bitwise:** & | ^ ~ << >> >>> (unsigned right shift)
- **Assignment:** = += -= *= /= %= &= |= ^= <<= >>= >>>=
- **Ternary:** condition ? valueIfTrue : valueIfFalse
- **instanceof:** Checks object type at runtime

```
int x = (a > b) ? a : b;  // ternary
if (obj instanceof String s) { ... }  // pattern matching (Java 16+)
```

Q: Explain short-circuit evaluation in logical operators.

Answer:

Short-circuit evaluation means the second operand is only evaluated if necessary:

- **&& (AND):** If first operand is false, second is NOT evaluated (result is already false)
- **|| (OR):** If first operand is true, second is NOT evaluated (result is already true)

```
// Safe null check using short-circuit
if (str != null && str.length() > 0) { ... }
// & and | evaluate both operands always (bitwise, no short-circuit)
```

 Note: Use && and || for boolean logic. Use & and | only for bitwise operations or when you explicitly need both sides evaluated.

3.2 Control Flow

Q: What are the control flow statements in Java?

Answer:

- **Conditional:** if, if-else, if-else if-else, switch, switch expression (Java 14+)
- **Loops:** for, while, do-while, enhanced for-each
- **Jump:** break, continue, return, (labeled break/continue)

```
// Switch expression (Java 14+)
String result = switch (day) {
    case MONDAY, TUESDAY -> "Weekday";
    case SATURDAY, SUNDAY -> "Weekend";
    default -> "Unknown";
};
```

Q: What is the difference between break, continue, and return?

Answer:

- **break:** Exits the current loop or switch statement entirely

- **continue:** Skips the rest of the current iteration and moves to the next
- **return:** Exits the current method; optionally returns a value

```
for (int i = 0; i < 10; i++) {
    if (i == 3) continue;  // skip 3
    if (i == 7) break;     // stop at 7
    System.out.println(i); // prints 0,1,2,4,5,6
}
```

4. Arrays & Strings

4.1 Arrays

Q: What is an array in Java? How are arrays stored in memory?

Answer:

An array is a fixed-size, ordered collection of elements of the same data type. Arrays in Java are objects stored on the heap; the variable holds a reference to the array.

- **Declaration:** int[] arr; (preferred) or int arr[];
- **Creation:** int[] arr = new int[5]; (default-initialized to 0)
- **Initialization:** int[] arr = {1, 2, 3, 4, 5};
- **Multi-dimensional:** int[][] matrix = new int[3][4];
- **Length:** arr.length (field, not method — no parentheses)

```
int[] arr = new int[3];  // [0, 0, 0]
arr[0] = 10; arr[1] = 20; arr[2] = 30;
// ArrayIndexOutOfBoundsException if index >= arr.length
```

> **Note:** Arrays.sort(arr) for sorting; Arrays.copyOf(arr, newLen) for copying; Arrays.fill(arr, val) to fill.

Q: What is a jagged array vs a multi-dimensional array?

Answer:

- **Multi-dimensional (rectangular):** int[3][4] — all rows have same length
- **Jagged array:** Each row can have different lengths

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];
```

4.2 Strings

Q: What is the String class in Java? What is the String Pool?

Answer:

String is an immutable class in java.lang. Once created, a String object's value cannot be changed.

- **String Pool (Interning):** A special area in the heap (Metaspace in Java 8+) where string literals are cached
  - String s = "hello" — stored in string pool; reused if same literal exists
  - String s = new String("hello") — creates a new object on heap, not in pool
- **Immutability benefits:** Thread-safe, hashCode caching, security, String pool efficiency

```
String s1 = "hello";
String s2 = "hello";
s1 == s2      // true — same pool reference
s1.intern()   // returns pool reference
```

Q: What is the difference between String, StringBuilder, and StringBuffer?

Answer:

Feature	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Performance	Slow (concatenation)	Fastest	Slower than SB
Use Case	Constants, literals	Single-thread ops	Multi-thread ops

```
StringBuilder sb = new StringBuilder();
sb.append("Hello").append(" ").append("World");
String result = sb.toString();  // "Hello World"
```

Q: What are important String methods you should know?

Answer:

- **length():** Returns number of characters
- **charAt(i):** Returns char at index i
- **substring(start, end):** Returns substring [start, end)
- **indexOf(str):** First occurrence index (-1 if not found)
- **contains(seq):** Boolean check
- **toLowerCase() / toUpperCase():** Case conversion
- **trim() / strip():** Remove whitespace (strip() handles Unicode whitespace)
- **replace(old, new):** Replace all occurrences
- **split(regex):** Returns String[] split by pattern
- **startsWith() / endsWith():** Prefix/suffix check
- **String.format() / formatted():** Printf-style formatting
- **String.join(delim, elements):** Join strings with delimiter

```
String s = "  Hello World  ";
s.trim()              // "Hello World"
s.strip()             // "Hello World" (Unicode-aware)
s.toLowerCase()       // "  hello world  "
String.join("-", "a", "b", "c")  // "a-b-c"
```

5. Exception Handling

5.1 Exception Hierarchy & Types

Q: What is the Java exception hierarchy?

Answer:

All exceptions in Java derive from the Throwable class:

- **Throwable**
  - Error — serious JVM problems; should NOT be caught (e.g., OutOfMemoryError, StackOverflowError)
  - Exception — application-level problems; should be handled
  - RuntimeException (Unchecked) — programming errors; not required to be caught
  - Other Exceptions (Checked) — must be declared or caught at compile time

Q: What is the difference between checked and unchecked exceptions?

Answer:

Aspect	Checked Exception	Unchecked Exception
Also called	Compile-time exception	Runtime exception
Compiler check	Must catch or declare (throws)	No requirement
Extends	Exception (not RuntimeException)	RuntimeException
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArrayIndexOutOfBoundsException, ClassCastException
Use when	Recoverable conditions (file not found)	Programming errors (logic bugs)

Q: Explain try-catch-finally and try-with-resources.

Answer:

```
// Traditional try-catch-finally
try {
    // code that may throw
} catch (IOException e) {
    // handle specific exception
} catch (Exception e) {
    // handle general exception
} finally {
    // always executes (even if return/exception)
}
```

```
// try-with-resources (Java 7+) — AutoCloseable resources
try (FileReader fr = new FileReader("file.txt");
     BufferedReader br = new BufferedReader(fr)) {
    String line = br.readLine();
} // fr and br automatically closed, even if exception
```

 Note: finally always executes EXCEPT if System.exit() is called or JVM crashes. try-with-resources calls close() before finally.

Q: What is the difference between throw and throws?

Answer:

- **throw:** Used inside a method body to explicitly throw an exception instance
- **throws:** Used in method signature to declare what checked exceptions a method may throw

```
public void readFile(String path) throws IOException {  // declares
    if (path == null) {
        throw new IllegalArgumentException("Path is null");  // throws
    }
    // ...
}
```

Q: How do you create a custom exception?

Answer:

```
// Checked custom exception
public class InsufficientFundsException extends Exception {
    private double amount;
    public InsufficientFundsException(double amount) {
        super("Insufficient funds: " + amount);
        this.amount = amount;
    }
    public double getAmount() { return amount; }
}
```

```
// Unchecked custom exception
public class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String msg) { super(msg); }
}
```

6. Collections Framework

6.1 Collections Overview

Q: What is the Java Collections Framework? Describe its hierarchy.

Answer:

The Java Collections Framework (JCF) provides a unified architecture for storing and manipulating groups of objects. Key interfaces:

- **Collection (root interface):**
  - List — ordered, allows duplicates (ArrayList, LinkedList, Vector, Stack)
  - Set — no duplicates (HashSet, LinkedHashSet, TreeSet)
  - Queue — FIFO ordering (LinkedList, PriorityQueue, ArrayDeque)
  - Deque — double-ended queue (ArrayDeque, LinkedList)
- **Map (NOT extends Collection):** Key-value pairs — no duplicate keys (HashMap, LinkedHashMap, TreeMap, Hashtable)

Q: What is the difference between ArrayList and LinkedList?

Answer:

Aspect	ArrayList	LinkedList
Internal Structure	Dynamic array	Doubly linked list
Access by index	O(1) — fast random access	O(n) — must traverse
Insert/Delete (middle)	O(n) — shift elements	O(1) if node reference known
Insert/Delete (end)	O(1) amortized	O(1)
Memory	Less (array)	More (node pointers)
Use when	Frequent reads, rare inserts	Frequent inserts/deletes

Q: What is the difference between HashMap, LinkedHashMap, and TreeMap?

Answer:

Feature	HashMap	LinkedHashMap	TreeMap
Order	No order	Insertion order	Sorted (natural or Comparator)
Null keys	1 null key	1 null key	No null keys
Performance	O(1) avg	O(1) avg	O(log n)
Thread-safe	No	No	No
Implements	Map	Map	NavigableMap, SortedMap

Q: Explain HashMap internal working — how does it handle collisions?

Answer:

- **Internal Structure:** Array of Node<K,V>[] (buckets). Default capacity: 16, load factor: 0.75
- **put(key, value) flow:**
  - 1. Compute hash(key.hashCode())
  - 2. index = hash & (capacity - 1) → find bucket
  - 3. If bucket empty → insert node
  - 4. If collision → chain (Linked List or Red-Black Tree if size > 8, Java 8+)

- **get(key) flow:**
  - Compute hash → find bucket → traverse chain using equals()
- **Resize (rehash):** When size > capacity × loadFactor; capacity doubles

> **Note:** Java 8+ converts a bucket's linked list to a Red-Black Tree when bucket size exceeds 8 (TREEIFY_THRESHOLD), improving worst-case lookup from O(n) to O(log n).

Q: What is the difference between HashSet and TreeSet?

Answer:

- **HashSet:** Backed by HashMap; O(1) add/remove/contains; unordered; allows 1 null
- **LinkedHashSet:** Backed by LinkedHashMap; maintains insertion order; O(1) operations
- **TreeSet:** Backed by TreeMap (Red-Black Tree); sorted; O(log n); no null

> **Note:** For Sets, elements must implement hashCode() and equals() correctly (HashSet) or be Comparable / use Comparator (TreeSet).

Q: What is the difference between Iterator and ListIterator?

Answer:

- **Iterator:** Traverses any Collection forward only; supports remove()
- **ListIterator:** For Lists only; traverses forward AND backward; supports add(), set(), remove()

```
List<String> list = new ArrayList<>(Arrays.asList("a","b","c"));
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("b")) it.remove();  // safe removal during iteration
}
```

> **Note:** Never modify a Collection while iterating with for-each — causes ConcurrentModificationException. Use Iterator.remove() or use CopyOnWriteArrayList for concurrent scenarios.

7. Generics

Q: What are Generics in Java and why are they used?

Answer:

Generics enable type-safe code by allowing classes, interfaces, and methods to operate on parameterized types. Introduced in Java 5.

- **Type Safety:** Compile-time type checking eliminates ClassCastException at runtime
- **Code Reuse:** Write one class/method that works with multiple types
- **No casting needed:** Compiler handles type casting automatically

```
// Without generics (raw type — unsafe)
List list = new ArrayList();
list.add("hello"); list.add(42);  // no error but risky
String s = (String) list.get(1);  // ClassCastException at runtime!
```

```
// With generics (type-safe)
List<String> list = new ArrayList<>();
list.add("hello");
// list.add(42);  // compile error
String s = list.get(0);  // no cast needed
```

Q: What is type erasure in Java Generics?

Answer:

Type erasure is the process by which the Java compiler removes all generic type information at compile time, replacing type parameters with their bounds (or Object if unbounded). The JVM has no knowledge of generics at runtime.

- Allows generics to work with pre-generic legacy code (backward compatibility)
- Generic type info is NOT available via reflection at runtime
- Cannot create generic arrays: new T[] is illegal

```
List<String> strList = new ArrayList<>();
List<Integer> intList = new ArrayList<>();
strList.getClass() == intList.getClass()  // true! Both are ArrayList.class
```

Q: What are bounded wildcards in Generics? Explain ? extends T and ? super T.

Answer:

- **? (Unbounded wildcard):** List<?> — unknown type; read-only essentially
- **? extends T (Upper bound):** List<? extends Number> — can read as Number; cannot add
- **? super T (Lower bound):** List<? super Integer> — can add Integer; reads as Object

Producer Extends Consumer Super (PECS) principle:

```
// PECS: Producer uses extends (you read/produce from it)
void print(List<? extends Number> list) { ... }
```

```
// Consumer uses super (you write/consume into it)
void addNumbers(List<? super Integer> list) { list.add(42); }
```

8. Multithreading & Concurrency

8.1 Thread Basics

Q: What is a thread? How do you create threads in Java?

Answer:

A thread is the smallest unit of execution within a process. Java supports multithreading natively. Two main ways to create threads:

- **Extend Thread class:**

```
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();
```

- **Implement Runnable interface (preferred — allows extending other classes):**

```
Runnable r = () -> System.out.println("Thread running");
new Thread(r).start();
```

- **Using ExecutorService (best practice for production):**

```
ExecutorService exec = Executors.newFixedThreadPool(4);
exec.submit(() -> System.out.println("Task running"));
exec.shutdown();
```

> **Note:** Always prefer Runnable/Callable with ExecutorService over raw Thread creation. Never call run() directly — always call start() to create a new thread.

Q: What are the thread states (Thread Lifecycle)?

Answer:

- **NEW:** Thread created but start() not called
- **RUNNABLE:** start() called; ready to run or currently running
- **BLOCKED:** Waiting to acquire a monitor lock (synchronized block)
- **WAITING:** Waiting indefinitely (wait(), join(), LockSupport.park())
- **TIMED_WAITING:** Waiting with timeout (sleep(n), wait(n), join(n))
- **TERMINATED:** run() method completed or exception thrown

Q: What is synchronization and why is it needed?

Answer:

Synchronization prevents race conditions by ensuring only one thread can execute a critical section at a time. Without it, concurrent access to shared mutable state causes data corruption.

```
// synchronized method
public synchronized void increment() { count++; }
```

```
// synchronized block (more granular — better performance)
public void increment() {
    synchronized (this) { count++; }
}
```

> **Note:** Synchronization has performance cost. Prefer java.util.concurrent classes (AtomicInteger, ConcurrentHashMap, etc.) for better performance in concurrent code.

Q: What is the difference between wait(), notify(), and sleep()?

Answer:

Aspect	wait() / notify()	sleep()
Class	Object	Thread
Lock release	Yes — releases monitor lock	No — holds lock while sleeping
Wake up by	notify() / notifyAll() / timeout	timeout only (or interrupt)
Must be in	synchronized block/method	Anywhere
Purpose	Inter-thread communication	Pause execution temporarily

Q: What is volatile keyword and when do you use it?

Answer:

The volatile keyword ensures that reads/writes to a variable are always done from/to main memory — not from a thread's local CPU cache. This guarantees visibility of changes across threads.

- Ensures visibility (not atomicity)
- Prevents CPU caching of the variable
- Does NOT make compound operations (i++) atomic

```
private volatile boolean running = true;
// Thread 1: running = false;
// Thread 2: while (running) { ... }  // sees change immediately
```

> **Note:** For atomicity, use AtomicBoolean, AtomicInteger, AtomicReference from java.util.concurrent.atomic.

Q: What is deadlock and how do you prevent it?

Answer:

Deadlock occurs when two or more threads are blocked forever, each waiting for the other to release a resource.

- **Prevention strategies:**
  - Lock ordering: Always acquire locks in the same fixed order across all threads
  - Lock timeout: Use tryLock(timeout) from ReentrantLock
  - Avoid nested locks: Minimize synchronized blocks
  - Use higher-level concurrency utilities (java.util.concurrent)

```
// Using ReentrantLock with timeout to prevent deadlock
if (lock1.tryLock(1, TimeUnit.SECONDS)) {
    if (lock2.tryLock(1, TimeUnit.SECONDS)) {
        // do work
    } else { lock1.unlock(); }
}
```

9. Java 8+ Features

9.1 Lambda Expressions & Functional Interfaces

Q: What is a lambda expression? What is a functional interface?

Answer:

A lambda expression is an anonymous function — a concise way to represent a single-method interface (functional interface) implementation.

- **Functional Interface:** An interface with exactly ONE abstract method (can have default/static methods). Annotated with @FunctionalInterface
- **Lambda syntax:** (params) -> expression OR (params) -> { statements; }

```
// Without lambda
Comparator<String> c = new Comparator<String>() {
    public int compare(String a, String b) { return a.compareTo(b); }
};
```

```
// With lambda
Comparator<String> c = (a, b) -> a.compareTo(b);
```

```
// Common functional interfaces (java.util.function)
Predicate<T>    // T -> boolean (test)
Function<T,R>   // T -> R (apply)
Consumer<T>     // T -> void (accept)
Supplier<T>     // () -> T (get)
BiFunction<T,U,R> // (T,U) -> R
```

Q: What is the Stream API? Explain intermediate vs terminal operations.

Answer:

The Stream API (java.util.stream) enables functional-style operations on collections. Streams are lazy — intermediate operations are not executed until a terminal operation is invoked.

- **Intermediate (lazy, return Stream):** filter(), map(), flatMap(), sorted(), distinct(), limit(), skip(), peek()
- **Terminal (eager, produce result):** collect(), forEach(), count(), reduce(), findFirst(), anyMatch(), allMatch(), min(), max(), toList()

```
List<String> result = names.stream()
    .filter(s -> s.startsWith("A"))     // intermediate
    .map(String::toUpperCase)            // intermediate
    .sorted()                            // intermediate
    .collect(Collectors.toList());       // terminal
```

```
// reduce example
int sum = List.of(1,2,3,4,5).stream()
    .reduce(0, Integer::sum);  // 15
```

Q: What is Optional in Java 8?

Answer:

Optional<T> is a container that may or may not contain a non-null value. It's designed to avoid NullPointerExceptions and make null handling explicit.

```
Optional<String> opt = Optional.ofNullable(getName()); // might be null
opt.isPresent()            // false if empty
opt.get()                  // throws if empty — avoid!
opt.orElse("default")      // return value or default
opt.orElseGet(() -> compute()) // lazy default
opt.orElseThrow()           // throw NoSuchElementException if empty
opt.map(String::toUpperCase) // transform if present
opt.filter(s -> s.length() > 3) // filter
```

Note: Don't use Optional as method parameter or field. Use it only as a return type to signal potential absence of a value.

Q: What are method references in Java 8?

Answer:

Method references are shorthand for lambdas that call an existing method. Syntax: ClassName::methodName

- **Static method:** Math::abs → x -> Math.abs(x)
- **Instance method (specific object):** str::toUpperCase → () -> str.toUpperCase()
- **Instance method (arbitrary object):** String::toUpperCase → s -> s.toUpperCase()
- **Constructor reference:** ArrayList::new → () -> new ArrayList<>()

```
List.of("a","b").stream().forEach(System.out::println);
List.of(-1,2,-3).stream().map(Math::abs).collect(Collectors.toList());
```

Q: What are default and static methods in interfaces (Java 8)?

Answer:

- **Default methods:** Concrete methods in interfaces (with 'default' keyword); allow interface evolution without breaking implementations
- **Static methods:** Utility methods belonging to the interface itself; not inherited by implementing classes

```
interface Vehicle {
    void start();                              // abstract
    default void stop() { System.out.println("Stopping"); }  // default
    static Vehicle create() { return new Car(); }  // static
}
```

Note: Java 9 added private methods in interfaces for sharing code between default methods without exposing them.

10. File I/O & Serialization

Q: What are the main I/O classes in Java?

Answer:

- **Byte Streams:** InputStream/OutputStream hierarchy — FileInputStream, FileOutputStream, BufferedInputStream
- **Character Streams:** Reader/Writer hierarchy — FileReader, FileWriter, BufferedReader, PrintWriter
- **NIO (New I/O — Java 7+):** java.nio.file — Files, Paths, Path — more efficient file operations

```
// Read all lines with NIO (Java 11+)
List<String> lines = Files.readAllLines(Path.of("file.txt"));
String content = Files.readString(Path.of("file.txt"));
```

```
// Write with NIO
Files.writeString(Path.of("out.txt"), "Hello", StandardOpenOption.CREATE);
```

Q: What is Serialization in Java?

Answer:

Serialization is the process of converting an object's state to a byte stream for storage or transmission. Deserialization is the reverse.

- **Implement Serializable:** Marker interface — no methods; just signals JVM
- **serialVersionUID:** Version control for serialized classes; should always be defined
- **transient keyword:** Fields marked transient are NOT serialized

```
public class Employee implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private transient String password;  // not serialized
}
```

```
// Serialize
ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("emp.ser"));
oos.writeObject(employee);
```

```
// Deserialize
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("emp.ser"));
Employee emp = (Employee) ois.readObject();
```

11. Memory Management & Garbage Collection

Q: How does Garbage Collection work in Java?

Answer:

The Java Garbage Collector automatically reclaims memory used by objects that are no longer reachable (no references pointing to them). The GC runs in the background and cannot be forced (System.gc() is only a hint).

- **Generational GC:** Heap divided into Young (Eden + Survivor) and Old generations
  - Young generation: New objects created here; frequent minor GCs
  - Old generation: Long-lived objects promoted here; infrequent major GCs
  - Metaspace (Java 8+): Class metadata; replaces PermGen
- **GC Algorithms:**
  - Serial GC — single thread; good for small apps
  - Parallel GC — multi-thread; throughput-focused (default in Java 8)
  - G1 GC — region-based; predictable pause times (default in Java 9+)
  - ZGC / Shenandoah — low-latency GCs; sub-millisecond pauses (Java 11+)

Q: What is a memory leak in Java? How do you prevent it?

Answer:

A memory leak occurs when objects are no longer needed but still referenced, preventing GC from reclaiming them.

- **Common causes:**
  - Static references holding large objects
  - Unclosed resources (connections, streams) — use try-with-resources
  - Listener/Observer not removed after use
  - ThreadLocal variables not cleaned up
  - Caches without eviction policy — use WeakHashMap or size-bounded caches
- **Prevention:**
  - Close resources in finally or try-with-resources
  - Use WeakReference/SoftReference for caches
  - Remove listeners when no longer needed
  - Profile with tools: VisualVM, Eclipse MAT, YourKit

Q: What is the difference between final, finally, and finalize?

Answer:

- **final:** Keyword — makes variable constant, method non-overridable, class non-inheritable
- **finally:** Block in try-catch — always executes for cleanup
- **finalize():** Method called by GC before object is collected (deprecated in Java 9, removed in Java 18) — AVOID using

```
final int MAX = 100;                    // constant
final class String { ... }              // cannot extend
try { ... } finally { connection.close(); }  // cleanup
```

12. Frequently Asked Interview Cheat Sheet ★

Quick Reference for the Most Common Java Interview Questions

★ JVM & Platform

JDK vs JRE vs JVM	JDK (develop) ⊃ JRE (run) ⊃ JVM (execute bytecode)
How Java is platform-independent	Source → javac → bytecode → JVM (platform-specific)
JIT compilation	Converts hot bytecode to native code at runtime for speed
Stack vs Heap	Stack: thread-local, primitives/refs, LIFO. Heap: shared, all objects

★ Data Types & Strings

Primitives count	8: byte, short, int, long, float, double, char, boolean
int default value	0 (fields); must initialize local variables
Integer cache range	-128 to 127 (use .equals() for comparison always)
String immutability	Cannot change content; creates new object on modification
String vs StringBuilder	String: immutable, thread-safe. StringBuilder: mutable, fast, not thread-safe
StringBuffer vs StringBuilder	StringBuffer: synchronized (thread-safe). StringBuilder: not synchronized (faster)
String Pool	Literals cached in pool; == works for same literal, use .equals() always

★ Exceptions

Checked exception	Must catch or declare; extends Exception (not RuntimeException)
Unchecked exception	Extends RuntimeException; not required to handle at compile time
Error vs Exception	Error: JVM issues, don't catch. Exception: application issues, handle them
finally execution	Always runs except System.exit() or JVM crash
try-with-resources	Auto-closes AutoCloseable; close() called before finally
throw vs throws	throw: throw instance in body. throws: declare in method signature

★ Collections

ArrayList vs LinkedList	ArrayList: O(1) access. LinkedList: O(1) insert/delete at known position
HashMap initial capacity	16 buckets, 0.75 load factor; rehashes when 75% full
HashMap collision (Java 8+)	LinkedList in bucket; converts to Red-Black Tree at 8 entries
HashMap vs Hashtable	HashMap: not synchronized, allows null key. Hashtable: synchronized, no null
HashMap vs ConcurrentHashMap	ConcurrentHashMap: segment-level locking, thread-safe, no null key/value
HashSet internal	Backed by HashMap; element is key, dummy object is value
Fail-fast vs fail-safe	Fail-fast (ArrayList, HashMap): throws

	ConcurrentModificationException. Fail-safe (CopyOnWriteArrayList): works on clone

★ Java 8+ Features

Functional Interface	Interface with exactly 1 abstract method (@FunctionalInterface)
Lambda syntax	(params) -> expression OR (params) -> { body; }
Stream intermediate ops	filter, map, flatMap, sorted, distinct, limit, skip (lazy)
Stream terminal ops	collect, forEach, count, reduce, findFirst, anyMatch (eager)
Optional.orElse vs orElseGet	orElse: always evaluates default. orElseGet: lazy (only if empty)
Default methods in interface	Concrete method in interface; allows backward-compatible evolution

★ Multithreading

Create thread ways	Extend Thread, implement Runnable, use ExecutorService (preferred)
start() vs run()	start(): creates new thread. run(): executes in current thread (no new thread)
volatile keyword	Ensures visibility across threads; no atomicity guarantee
synchronized keyword	Mutual exclusion; one thread at a time in block/method
wait() vs sleep()	wait(): releases lock, Object class. sleep(): holds lock, Thread class
Deadlock prevention	Lock ordering, tryLock with timeout, use java.util.concurrent
volatile vs AtomicInteger	volatile: visibility. AtomicInteger: visibility + atomicity (CAS operations)

★ Memory & GC

Generations in heap	Young (Eden + Survivor S0/S1) + Old + Metaspace
Minor GC vs Major GC	Minor: Young gen (frequent, fast). Major: Old gen (infrequent, slow)
final vs finally vs finalize	final: constant/non-override. finally: cleanup. finalize: deprecated GC hook
Prevent memory leaks	Close resources, remove listeners, avoid static refs, use WeakReference
System.gc()	Hint only — JVM may ignore it; cannot force GC

★ Miscellaneous Must-Knows

== vs .equals()	==: reference equality. equals(): logical/content equality
hashCode contract	Equal objects must have equal hashCodes; override both together
Comparable vs Comparator	Comparable: natural order (compareTo). Comparator: custom order (compare)
var keyword (Java 10+)	Local variable type inference; type still static, inferred by compiler
Record (Java 16+)	Immutable data class with auto constructor, getters, equals, hashCode, toString
Sealed classes (Java 17+)	Restricts which classes can extend/implement the sealed class

Good Luck with Your Interview! ✔

Remember: Understand concepts deeply, not just memorize answers.